

Quality Assurance

Quality Assurance

Quality Assurance

The systematic process of ensuring that an integrated system or acquired component meets specified organizational standards, compliance laws, and stakeholder expectations.

Focuses on *preventing defects* throughout the design and integration lifecycle, rather than just catching bugs at the very end.

- Establishes *procedural governance* to guarantee that both internal teams and external vendors adhere to the same operational baselines.

Guidelines Development

Building a comprehensive, standardized playbook that dictates the rules of engagement for developers, architects, and third-party vendors.

- *Integration Standards*: Mandating specific data transfer formats (e.g., JSON/XML payloads), naming conventions, and API design rules (like RESTful constraints).
- *Security Baselines*: Forcing specific authentication policies (e.g., OAuth 2.0, HTTPS encryption, API gateway keys) onto any module plugged into the corporate infrastructure.
- *Documentation Templates*: Standardizing how system topologies, dependencies, and interface architectures must be documented for long-term maintenance.

ISO/IEC 25010: The Quality Model

ISO/IEC 25010 is the international standard for evaluating software product quality. It provides a *structured framework* for defining and measuring quality requirements.

- *Product Quality Model*: Defines 9 characteristics:
 - Functional Suitability,
 - Performance Efficiency,
 - Compatibility,
 - Interaction Capability,
 - Reliability,
 - Security,
 - Maintainability,
 - Flexibility, and
 - Safety.
- *Quality in Use Model*: Evaluates how the software performs for the actual end-user
 - Effectiveness,
 - Efficiency,
 - Satisfaction,
 - Freedom from Risk, and
 - Context Coverage

Capability Maturity Model (CMMI)

A framework used to improve an organization's processes and its ability to manage the development, acquisition, and maintenance of products and services.

The CMMI works off of *maturity* levels, which are:

1. *Initial*: Processes are unpredictable and reactive.
2. *Managed*: Processes are planned and documented at the project level.
3. *Defined*: Processes are standardized across the entire organization.
4. *Quantitatively Managed*: Processes are measured and controlled using data.
5. *Optimizing*: Continuous process improvement.

Applicable to both software development and broader organizational management.

ISO/IEC 27001: Information Security

The international standard for establishing, implementing, maintaining, and continually improving an *Information Security Management System (ISMS)*.

This standard's goal is to protect the *CIA Triad* (Confidentiality, Integrity, and Availability) of organizational data.

Vendors are often required to be ISO 27001 certified before they can be considered in an *RFP* process, ensuring they have robust security controls in place.

The Cost of Certification

Most smaller businesses don't require extensive certification to accept a proposal.

Certification is an *extensive* process which can cost *up to* 75,000 USD (4mil php), and hundreds of hours of training and management.

Review Mechanisms

Structured, **repeatable** audit processes used to evaluate and verify project deliverables at critical stages of the integration lifecycle.

- *Architectural Walkthroughs*: Peer reviews of *system blueprints*, *data flow diagrams* (DFDs), and requirement *traceability* matrices before code is written.
- *Code and Interface Inspections*: Checking custom middleware, integration code, and wrapper scripts for security flaws and performance leaks.
- *Phase-Gate Reviews*: Formal "Go/No-Go" governance milestones at the end of project phases (e.g., preventing an outsourced module from moving into the testing environment until it passes the sourcing review).

Quality Metrics and Checkpoints

The quantitative baselines and gatekeeper phases used to systematically measure system maturity and authorize final deployment.

- *Key Quality Metrics:*

- **Defect Density:** Tracking the number of operational bugs per module.
- **Test Coverage:** The percentage of code verified by automated test suites.
- **Performance Drift:** Measuring response time and resource changes against the original baselines.

- *Critical Checkpoints:*

- **The Design Gate:** Ensuring requirements are traceable and legally sound.
- **The Integration Gate:** Verifying multi-module builds pass baseline functionality.
- **The Security Gate:** Auditing compliance before public rollout.